# Let's implement useless Python objects

Hayao Suzuki

PythonAsia 2026 at DE LA SALLE UNIVERSITY, MALATE, MANILA

March 21, 2026

# Share it

## GitHub

- https://github.com/HayaoSuzuki/pythonasia2026

## Hashtag

- #PythonAsia2026

# Who am I ?

## About me

Name Hayao Suzuki（鈴木　駿）

~~Twitter~~ X @CardinalXaro

Work Software Engineer at Tokyo Gas CO.,LTD

Fun Playing tuba in the Tokyo Gas Wind Orchestra

## About Tokyo Gas

- The largest natural gas utility in Japan
- Production and sale of city gas, LNG sales, etc.
- Gas Pipeline Service Business、Supply of city gas
- Production, supply and sale of electricity

# Who am I ?

## Presentations of PyCon

- Tomorrow graphlib, Let Us Use Everybody(PyCon JP 2025)
- Let's Implement Useless Python Objects(PyCon APAC 2023)
- Unknown Evolution of the Built-in Function pow(PyCon JP 2021)
- How to Use In-Memory Streams(PyCon JP 2020)
- Do You Know cmath Module?(PyCon mini Shizuoka 2020)
- Elementary Number Theory with Python(PyCon mini Hiroshima 2019)
- Symbolic Mathematics Using SymPy(PyCon JP 2018)

Listed at **https://xaro.hatenablog.jp/**

# Today's Theme

Let's implement useless Python objects

# What is it mean useless?

## From Longman Dictionary of Contemporary English

1. not useful or effective in any way
2. (informal) unable or unwilling to do anything properly

# Is the useless object really useless?

**From Zhuangzi "The Human World" (莊子 人間世篇)**

人皆知有用之用 而莫知無用之用也

Everyone knows the usefulness of the useful, but no one knows the usefulness of the useless.

# Today's Theme

## Let's implement useless Python objects

The useless objects are useless, but how to make a useless object is very useful.

# What is a useless Python object?

**Example: LiarContainer**

```
>>> c = LiarContainer(["spam", "egg", "bacon"])
>>> "spam" in c
False
>>> "tomato" in c
True
```

# What is a useless Python object?

**Example: FibonacciSized**

```
>>> s = FibonacciSized(range(50))
>>> len(s)
12586269025
```

# What is a useless Python object?

## Example: ShuffledIterable

```
>>> it = ShuffledIterable([1, 2, 3, 4, 5])
>>> for _ in range(3):
...     for v in it:
...         print(v, end=" ")
...     print()
...
5 3 4 2 1
4 1 2 3 5
2 5 3 1 4
```

# What is a useless Python object?

## Definition of a useless Python object in this talk

A useless Python object behave Pythonic, but does not work as expected.

# Data Structures and Operations

## Basic Data Structures of Python

| | |
|---|---|
| List | `[1, 4, 9, 16, 25, 36]` |
| Tuple | `("pen", "pineapple", "apple", "pen")` |
| Dictionary | `{"Answer": 42}` |
| Set | `{41, 43, 47, 53, 57, 59}` |

## Common Operations of Data Structure

| | |
|---|---|
| `len()` | Length of object |
| `in` | Membership test |
| `for` | Iteration |

# in and Container

## object.__contains__()

Called to implement membership test operators.

## Example: LiarContainer

```
class LiarContainer(Container):
    def __contains__(self, item) -> bool:
        return item not in self._data
```

# len() and Sized

## object.__len__()

Called to implement the built-in function **len()**.

## Example: FibonacciSized

```
class FibonacciSized(Sized):
    PHI: Final[float] = (1 + math.sqrt(5)) / 2
    def __len__(self) -> int:
        return round(
            (1 / math.sqrt(5))
            * pow(self.PHI, len(self._data))
        )
```

# for and Iterable

## object.__iter__()

Called when an iterator is required for a container.

## Example: ShuffledIterable

```
class ShuffledIterable(Iterable):
    def __iter__(self) -> Iterator:
        return iter(
            random.sample(self._data, k=len(self._data))
        )
```

# Object Protocols

## How to implement Pythonic Python objects

We need to understand object protocols.

Ref: `https://docs.python.org/3/reference/datamodel.html`

# collections.abc

## collections.abc

This module provides abstract base classes that can be used to test whether a class provides a particular interface.

From
https://docs.python.org/3/library/collections.abc.html

# collections.abc

## collections.abc and Interface

| ABC | Interface |
|---|---|
| Sized | __len__() |
| Container | __contains__() |
| Iterable | __iter__() |
| Collection | Sized, Container, Iterable |

# Collection

## collections.abc.Collection

Sized and Container and Iterable

## Example: Collection

```
class UselessCollection(
    FibonacciSized, LiarContainer, ShuffledIterable
):
    pass
```

# collections.abc

## collections.abc and Built-in Objects

| ABC | built-in objects |
|---|---|
| Sequence | tuple |
| MutableSequence | list |
| MutableSet | set |
| MutableMapping | dict |

# Sequence

## Example: ModularSequence

```
class ModularSequence(Sequence):
    def __getitem__(self, key):
        match key:
            case int():
                return self._data[key % len(self._data)]
            case slice():
                n = len(self._data)
                s = slice(
                    key.start % n, key.stop % n, key.step
                )
                return self._data[s]
            case _:
                raise TypeError
```

# Sequence

## Example: ModularSequence

```
>>> seq = ModularSequence(range(20))
>>> print(seq[21:44])
[1, 2, 3]
>>> print(seq[65543])
3
>>> seq.count(13) # It does not stop.
```

# Sequence

## Example: CompetitionSequence

```
class CompetitionSequence(Sequence):
    def __getitem__(self, index):
        return self._data[index]

    def __iter__(self):
        return iter(reversed(self._data))

    def __len__(self):
        return len(self._data)
```

# Sequence

## Example: CompetitionSequence

```
>>> s = CompetitionSequence("abcdefg")
>>> s[0]
'a'
>>> list(s)
['g', 'f', 'e', 'd', 'c', 'b', 'a']
>>> len(s)
7
```

# Mapping

## Example: MisprintedDictionary

```
class MisprintedDictionary(Mapping):
    def __init__(self, _dict: dict):
        values = _dict.values()
        rotated_values = chain(
            islice(values, 1, None),
            islice(values, 1),
        )
        self._data = dict(
            zip(_dict, rotated_values)
        )
```

# Mapping

## Example: MisprintedDictionary

```
>>> d = MisprintedDictionary({"a": 1, "b": 2, "c": 3})
>>> d["a"], d["b"], d["c"]
(2, 3, 1)
>>> list(d.keys())
['a', 'b', 'c']
>>> len(d)
2
```

# Set

## Example: CrowdSet

```
@functools.total_ordering
class CrowdSet(Set):
    def __init__(self, data=None):
        if data is not None:
            self._data = set(data)
        else:
            self._data = set()

    def __lt__(self, other):
        return self._data >= other._data
```

# Set

## Example: CrowdSet

```
class CrowdSet(Set):
    def __len__(self):
        return len(self._data) ** 2

    def __and__(self, other):
        return CrowdSet(self._data | other._data)

    def __or__(self, other):
        return CrowdSet(self._data & other._data)
```

# Set

## Example: CrowdSet

```
>>> s = CrowdSet(("egg", "bacon", "spam"))
>>> t = CrowdSet(("egg", "egg", "spam", "spam"))
>>> s > t
True
>>> len(s)
9
>>> sorted(s & t)
['bacon', 'egg', 'spam']
>>> sorted(s | t)
['egg', 'spam']
```

# Conclusion

## Let's implement useless Python objects

- Useless Python objects are useful
- `collections.abc` module is very useful
- Once you understand object protocol, you can do anything